

Flexible Abacus VM: Evolutionary Bytecode Execution Framework

A lightweight, high-performance Python/NumPy implementation of a **stochastic Virtual Machine (VM)** coupled with a **Non-Gradient Evolutionary Strategy**.

Instead of relying on conventional backpropagation or standard neural network abstractions, *Flexible Abacus* executes programs within a custom 4D probabilistic state tensor ("the cart"). The state dynamically mutates over generations to solve numerical operations—such as fixed-point square root approximation—directly at the byte level.

Technical Highlights

- **Custom Instruction Set Architecture (ISA):** Implements an opcode execution loop featuring arithmetic execution blocks, dynamic state flippers, and direct buffer writers.
- **Probabilistic Instruction Fetching:** Next-instruction selection (WholsNext) uses non-deterministic weighted sampling based on probability points within state tensors.
- **Non-Gradient Optimization:** Mutates state matrices (NewChallenger) via localized integer drift and evaluates candidates using target objective functions without explicit gradient calculations.
- **Low-Level Numerical Manipulation:** Employs raw byte-level packing and unpacking (struct.unpack) alongside explicit multi-dimensional numpy array typing (np.uint8, np.uint32, np.uint64).

Architecture Overview

The system operates across three primary buffers: inTray (Input Buffer), outTray (Output Buffer), and the core state matrix referred to as the **Cart**.

Input Buffer (256 x uint8) (inTray)

Flexible Abacus VM

State Tensor (Cart)

Fetch Engine (WholsNext)

Weighted Random Sampling over Probability Points

Extraction & Blend Engine

Cross-shelf average extraction

Opcode Execution Pipeline

[0-63] Processor (ALU Math Operations)

[64-127] Flipper (Internal State Mut)
[128-191] Read (Import from inTray)
[192-255] Write (Export to outTray)

Memory Write-Back (Ink & Bounce Feedback)

Output Buffer (4 x uint8) (outTray)

Virtual Machine & Instruction Set Specification

Memory Map (cart)

The execution state is held in a 4D tensor of shape (4, 256, 11, 4) typed as np.uint8:

- **Dimension 0 (Shelves):** 4 parallel memory banks (s = [0..3]).
- **Dimension 1 (Books):** 256 addressable instruction slots per shelf (b = [0..255]).
- **Dimension 2 (Columns):** 11 registers containing control signals, opcode parameters, and probability weights (c = [0..10]).
- **Dimension 3 (Rows):** 4-element vectors containing payload values (r = [0..3]).

Dynamic Opcode Routing

During each tick, an instruction vector (extracted) is sampled across the 4 shelves and executed based on extracted[0, 1]:

Opcode Range	Function	Execution Description
0 - 63	Processor (ALU)	Performs arithmetic operations (/, -, +, *) between input parameters and scales intermediate results.
64 - 127	Flipper	Modifies internal state address coordinates within the cart tensor dynamically during execution.
128 - 191	Read Input	Reads a byte from inTray at index extracted[9, 3] and writes it to internal register extracted[6, 2].

192 - 255	Write Output	Writes the target value to outTray at index designated by control register bounds.
-----------	---------------------	--

Evolutionary Optimization Cycle

The algorithm searches for execution states capable of approximating arbitrary targets

(default: $f(x) = \sqrt{x}$):

1. **Initialization:** Instantiates baseline state tensor existing and state tracking files (experimentcart.npy, cartstats.npy).
2. **Evaluation:** Executes the VM over N clock cycles (ticks = 1000) on input data, mapping outputs through a fitness transform.
3. **Mutation:** Constructs a candidate challenger tensor NewChallenger via bounded random walk modification.
4. **Selection:** Evaluates candidate score $S_{\text{challenger}}$ against baseline S_{existing} . If superior, the baseline state is replaced and persisted to disk.

Getting Started

Prerequisites

Ensure you have Python 3.8+ installed along with the required numerical dependencies:

```
pip install numpy matplotlib
```

Execution

Run the simulation directly:

```
python "flexible abacus.py"
```

- **Interactive Mode:** The script renders real-time score convergence histograms using matplotlib.pyplot.
- **State Persistence:** Execution progress automatically persists state weights to experimentcart.npy and historical performance to cartstats.npy.

